

Understanding the BO3 to BO2 Apothicon Servant Port Problem

Detailed architecture and failure analysis for `pluto_t6_full_game`

Prepared from the current repository state plus external modding references

March 11, 2026

Contents

1	Purpose and the most important conclusion	2
2	What you are trying to do, in plain language	2
2.1	Useful terms	3
3	BO2/T6 and Plutonium architecture	3
3.1	The BO2/T6 asset stack	3
3.2	Plutonium mod and script loading	4
3.3	BO2 zombies weapon registration lifecycle	4
3.4	Script awareness versus engine grantability	5
4	BO3/T7 first-person weapon authoring architecture	5
4.1	What the BO3 IDG source bundle tells us	5
4.2	What the BO3 weapon definition implies	6
4.3	Why BO3 extraction pipelines matter	6
5	BO3 and BO2 compared directly	7
6	What your current repo is doing right now	8
6.1	Current GSC behavior	8
6.2	Current builder behavior	8
6.3	What the current fastfiles prove	9
7	The hard evidence from the current asset set	9
7.1	Current rig and animation counts	9

7.2	Adding BO3 hands still does not save it	9
7.3	What the T6 bone budget means	10
8	Why the current approach fails	10
8.1	Failure A: wrong registration timing	10
8.2	Failure B: mismatched rig contract	10
8.3	Failure C: literal BO3 animation import is not a valid endgame	11
8.4	Failure D: the current weapon is still semantically ray-gun-derived	11
8.5	Failure E: not enough runtime proof for the current fresh-name path	11
9	The correct mental model	12
9.1	You are solving three separate gates	12
9.2	What donor surfaces are actually for	12
9.3	What <code>viewModel_hands_no_model</code> really implies	12
10	What you are doing wrong today, stated bluntly	13
11	What the feasible path forward looks like	13
11.1	Short-term experiment plan	13
11.2	What the final architecture should probably be	13
11.3	Why your repo is already pointing in this direction	14
12	Recommended implementation philosophy	14
13	Summary	14
A	Local repository evidence used for this analysis	15
B	External references	15

1 Purpose and the most important conclusion

This document explains what is actually happening in your current BO3-to-BO2 weapon port attempt, why the current Apothicon Servant probe is failing, and what architecture you need in your head if you want to stop guessing and start making deliberate progress.

The most important conclusion is simple:

1. You are hitting **two different failures**, not one.
2. The current GSC rewrite cannot solve the deeper problem because it is trying to fix an **init-time registration problem** at **player-spawn time**.
3. Even if registration were fixed, the current model and animation pairing is still structurally invalid because the BO3 IDG animations target a rig that the current BO2-side model does not have.

That means this is **not** a case of “one missing bone” or “one bad script call.” It is an architecture mismatch between:

- BO2/T6 zombies weapon registration lifecycle
- BO2/T6 first-person rig limits and camera/tag contracts
- BO3/T7 first-person weapon authoring assumptions
- your current probe implementation, which mixes a fresh-name BO2 weapon experiment with a near-native BO3 animation experiment

2 What you are trying to do, in plain language

Your stated goal is to emit a *real* `apothicon_servant_zm` weapon on a BO2/Plutonium zombies runtime, using a donor weapon surface that behaves more like a ray-gun-family projectile weapon than like an assault rifle carrier. You also want the spawn logic reduced to only three responsibilities:

1. register the weapon
2. grant `m1911_zm`
3. grant `apothicon_servant_zm`

That goal is valid, but it contains several hidden assumptions:

- that “registering” a zombies weapon table entry is the same thing as engine-level weapon registration
- that if a weapon asset exists in a fastfile and a script table entry exists, `giveweapon()` should work
- that if BO3 animations compile into T6 `xanim` assets, they are probably safe to play on the imported BO3 model

- that a BO3 wonder weapon viewmodel can be made acceptable to BO2 by changing only a few `weapondef` fields

Those assumptions are the reason the work feels confusing. The engine is stricter than those mental models.

2.1 Useful terms

Fresh-name weapon. A truly new BO2/T6 weapon asset such as `apothicon_servant_zm`, not a renamed donor at the script level only.

Donor surface. A known-good weapon envelope whose fields, asset references, or state-machine expectations are borrowed as a compatibility surface. In your current probe that donor is ray-gun-like.

Truth-alias carrier. A stronger workaround than a donor surface. This means the engine still believes the weapon is a fully registered stock weapon, while you patch the model, anim, and behavior around it. Your repo previously used this idea for thundergun work.

Combined first-person rig. A single first-person animated model that already contains the weapon bones, hands/arms, and important tags like camera and ADS anchors. In T6, this matters because many weapons use `viewmodel_hands_no_model`, which means the gun model itself must satisfy the first-person contract.

3 BO2/T6 and Plutonium architecture

3.1 The BO2/T6 asset stack

At the asset level, BO2/T6 is not just “a model plus a script.” The important layers are:

1. raw asset data on disk
2. linked fastfiles that package those assets
3. engine-side asset loading and registration
4. map- and mode-specific script tables that reference those assets
5. runtime grant, switch, and fire behavior

OpenAssetTools documents that T6 zone files can emit not only `weapon`, but also `weapondef`, `weaponvariant`, and `weaponfull` assets.¹ That matters because it already tells you that “weapon” is only one part of a larger weapon-definition story.

Your local T6 headers reinforce this. In `\_tmp\_T6\_Assets.h`, the important structs are:

- `WeaponVariantDef`
- `WeaponDef`

¹<https://openassettools.dev/reference/zone-file.html>

- `WeaponFullDef`

And in `\_tmp\_ContentLoaderT6.cpp`, `ASSET_TYPE_WEAPON` is loaded as `WeaponVariantDef`. The practical implication is:

Seeing a weapon string in a fastfile, or even seeing a script table entry for that weapon, does **not** automatically mean the runtime has a fully usable weapon definition in the state required by `giveweapon()`.

3.2 Plutonium mod and script loading

Plutonium gives you mod-scoped fastfile loading and custom script loading, but those systems have their own ordering rules.

Plutonium documents that T6 scripts can live under `%localappdata%\textbackslashPlutonium\textbackslashstorage\textbackslashst6\textbackslashscripts`, and that custom T6 scripts need an `init()` or `main()` entry point.² Plutonium also documents newer scripting features such as `replaceFunc()` and explains that custom-script `main()` runs before stock scripts, specifically so you can patch behavior before the stock code consumes it.³

This directly matters for your case. If a stock system reads weapon tables during early map initialization, then trying to modify those tables later on `spawned_player` is the wrong lifecycle point.

Plutonium's changelog is also relevant here:

- unsigned T6 fastfiles can be loaded when a mod is loaded and `fs\_game` is populated⁴
- `weapon\_load\_order` exists so raw weapon assets can take precedence over fastfile-loaded ones⁵
- `dump\_assets` can dump weapon and attachment assets for debugging⁶

So Plutonium is not the blocker by itself. It gives you useful tools. But those tools do not remove T6's internal expectations.

3.3 BO2 zombies weapon registration lifecycle

This is one of the most important pieces of the whole analysis.

From the decompiled stock scripts in your repo:

- `decompiled\_scripts/ZM/Core/maps/mp/zombies/\_zm\_weapons.gsc` calls `init\_weapons()` during its own `init()`
- the map script, such as `decompiled\_scripts/ZM/Maps/Tranzit/maps/mp/zm\_transit.gsc`, sets up zombies-specific callbacks during `main()`

²<https://plutonium.pw/docs/modding/loading-mods/>

³<https://plutonium.pw/docs/modding/gsc/new-scripting-features/>

⁴<https://plutonium.pw/docs/changelog/>

⁵<https://plutonium.pw/docs/changelog/>

⁶<https://plutonium.pw/docs/changelog/>

- the normal player-spawn watcher in `\_zm\_weapons.gsc` is for player runtime behavior, not for base weapon registration

That means BO2 zombies weapon setup is front-loaded into the early map boot process. The stock architecture expects weapon inclusion and custom addition hooks to be defined **before** the weapon system completes its initialization pass.

Your current mod script does this instead:

- wait for `spawned\_player`
- then call `include\_zombie\_weapon()`
- then call `add\_zombie\_weapon()`
- then clone donor metadata into `level.zombie\_weapons["apothicon\_servant\_zm"]`

This is almost certainly too late for the engine-side state that matters to `giveweapon()`.

3.4 Script awareness versus engine grantability

One of the easiest ways to get lost in T6 zombies modding is to blur these layers together:

1. **Asset exists in fastfile.**
2. **Script knows the weapon name.**
3. **Zombies weapon tables contain the weapon entry.**
4. **Engine can resolve a valid `weapondef`/variant for grant and switch.**

Those are not the same thing.

Your repo history already found this with the older `weapondef\_unregistered` failures. A weapon can pass softer checks such as “included” or “can use content” while still failing the real grant path because the deeper engine-side weapon registration did not occur in the required way.

4 BO3/T7 first-person weapon authoring architecture

4.1 What the BO3 IDG source bundle tells us

Your local BO3 source-side files are revealing:

- `source\_data/\_midgetblaster/t7\_idgun.gdt`
- `share/raw/zone\_source/idgun.zpkg`
- `share/raw/zone\_source/t7\_idgun\_genesis.zpkg`
- `xanim\_export/\_midgetblaster/black\_ops\_3/vm\_zod\_id\_gun\_*.xanim\_bin`

- `model\_export/\_midgetblaster/weapons/t7/wp\_t7\_zmb\_zod\_idg/wp\_t7\_zmb\_zod\_idg\_view.xmodel\_bin`

From the GDT and bundle contents, the BO3 IDG is not just a model replacement. It comes with:

- dedicated weapon definitions
- dedicated scripts
- dedicated first-person and third-person FX
- a dedicated first-person animation set

That is the first reason a true Apothicon Servant port is much harder than a simple BO2 donor-surface swap.

4.2 What the BO3 weapon definition implies

The BO3 GDT shows several facts that matter:

- `gunModel` is `wp\_t7\_zmb\_zod\_idg\_view`
- `viewmodelTag` is `tag\_weapon\_right`
- `worldModelTagRight` is `tag\_weapon\_right`
- `worldModelTagLeft` is `tag\_weapon\_left`
- `handModel` is empty
- it has dedicated muzzle, trail, and explosion FX
- `parentWeaponName` is empty in the source weapon

I infer from these facts that BO3's first-person authoring contract is not the same as BO2's. In BO3, the IDG asset is authored with an explicit viewmodel tag convention and a richer camera-authoring environment. CoDMayaTools explicitly advertises full BO3 support, a camera animation toolkit, and BO3 camera export support.⁷

That strongly suggests the BO3 authoring pipeline assumes a richer first-person rig/camera contract than the older BO2/T6 runtime.

4.3 Why BO3 extraction pipelines matter

HydraX describes itself as an asset decompiler for BO3 Mod Tools intended to export asset types Treyarch did not fully provide with the mod tools.⁸ This matters because it explains why BO3 source-side workflows are often assembled from multiple export and reverse-engineering tools rather than from a single official, clean source format.

That in turn means a BO3-to-BO2 port pipeline is often dealing with:

⁷<https://theenbywitch.github.io/CoDMayaTools/>

⁸<https://github.com/Scobalula/HydraX>

- exported data
- partially reconstructed source conventions
- authoring assumptions that were valid in T7 but not in T6

5 BO3 and BO2 compared directly

Topic	BO2/T6 reality	BO3/T7 reality
Weapon registration	Strongly tied to early map and mode initialization, especially in zombies. Script tables alone do not prove grantability.	Weapon assets and scripts are authored in a newer runtime with different assumptions; not directly portable by name.
First-person rig	Tight first-person DObj constraints; practical hard cap around 160 bones for gun plus hands in the combined first-person object.	BO3 wonder-weapon rigs and anim sets can assume larger combined first-person skeletons and richer tag/camera setups.
Hand model contract	Many weapons use <code>viewmodel_hands_no_model</code> ; when they do, the gun model must already satisfy the combined rig contract.	The IDG source weapon has empty <code>handModel</code> , which suggests a different authoring convention rather than BO2's exact combined-rig behavior.
Camera chain	<code>tag_view</code> , <code>tag_ads</code> , <code>tag_cambone</code> , and <code>tag_camera</code> are structural, not cosmetic. Bad hierarchy or bind rotations can flip view basis.	BO3 tooling explicitly supports dedicated camera animation export. That points to a more explicit first-person camera authoring pipeline.
Animation compatibility	T6 <code>XAnimParts</code> can compile and still fail semantically if the target rig is wrong, incomplete, or outside the expected channel set.	BO3 animations were authored against BO3 rig conventions and therefore need retargeting or rig reconstruction to behave in T6.
Packaging	T6 via OAT can emit <code>weapon</code> , <code>weapondef</code> , <code>weaponvariant</code> , and <code>weaponfull</code> ; fastfile presence alone is not the same as runtime acceptance.	BO3 source bundles often come through GDT, zone-source bundles, and extracted/exported assets with their own tooling ecosystem.
Gameplay parity	Donor surfaces can get you acceptance, but they are not parity.	The real IDG has dedicated FX and scripts that go beyond a ray-gun-like surface.

6 What your current repo is doing right now

6.1 Current GSC behavior

The current script in `mods/bo3\_rev/scripts/mod\_i\_am\_mod.gsc` has the behavior you described, but with an important caveat:

1. `init()` and `main()` both start the mod bootstrap
2. on player connect, the player gets a spawn watcher
3. on `spawned\_player`, after a short wait, the script:
 - precaches `apothicon\_servant\_zm`
 - precaches `bo3\_rev\_idg\_view`
 - calls `include\_zombie\_weapon()`
 - calls `add\_zombie\_weapon()`
 - clones donor metadata into `level.zombie\_weapons`
 - grants `m1911\_zm`
 - grants `apothicon\_servant\_zm`
 - switches to `apothicon\_servant\_zm`

So yes, the script has been simplified conceptually. But the simplification happened at the *wrong lifecycle point*. It is still a spawn-time patch, not an early zombies-boot integration.

6.2 Current builder behavior

The probe builder in `\_build/build\_bo3\_rev\_idg\_probe.py` stages a fresh-name weapon, but it does so by cloning a BO2 ray gun surface and then swapping selected fields.

The current staged weapon does all of the following:

- `gunModel=bo3\_rev\_idg\_view`
- `handModel=viewmodel\_hands\_no\_model`
- `worldModel=weapon\_usa\_ray\_gun`
- `ammoName=ray\_gun`
- `clipName=ray\_gun`
- `parentWeaponName=ray\_gun`
- projectile and impact FX remain ray-gun-family
- nearly every first-person animation field is pointed at `vm\_zod\_id\_gun\_*`

That means your current fresh-name weapon is not a pure IDG definition. It is a **ray-gun gameplay surface with an IDG model and IDG animation references**.

6.3 What the current fastfiles prove

The current build report shows that the mod fastfiles do successfully package:

- the IDG viewmodel xmodel
- the `vm\_zod\_id\_gun\_*` animation set
- the fresh-name `apothicon\_servant\_zm` weapon asset

So the problem is not “nothing is building.” Things are building. The question is whether they satisfy T6’s runtime contracts.

7 The hard evidence from the current asset set

7.1 Current rig and animation counts

The strongest technical evidence in the repo is the actual mismatch between the staged model and the BO3 animation set:

- `idg\_view.glb`: 143 nodes, 1 skin, 141 joints
- union of `vm\_zod\_id\_gun\_*` animation bone references: 251 named bones
- missing from the model relative to the animation union: 110 bones

Critical tags and anchors:

- `tag\_weapon`: present in the model and referenced by the anims
- `tag\_flash`: present in the model and referenced by the anims
- `tag\_view`: missing from the model
- `tag\_camera`: missing from the model
- `tag\_cambone`: missing from the model
- `tag\_torso`: missing from the model
- `tag\_weapon\_left`: missing from the model
- `tag\_weapon\_right`: missing from the model

This alone is enough to say the current model and animation set do not share the same skeleton contract.

7.2 Adding BO3 hands still does not save it

When the separate BO3 viewhands model is considered:

- `bo3\_richtofen\_viewhands.glb`: 68 nodes, 1 skin, 66 joints
- unique node union of IDG view plus BO3 hands: 211
- overlap between the two skeleton name sets: 0
- even after combining them, the animation union is still missing 45 bones

This is a devastating result for a literal port strategy, because:

1. the combined BO3-side first-person skeleton still does not cover the full animation contract
2. 211 is already far beyond the practical T6 first-person bone budget

7.3 What the T6 bone budget means

Your own repo documentation already captured the practical wall: a T6 first-person DObj can only tolerate about 160 bones total for gun plus hands. That is why earlier experiments hit errors like:

```
doobjforxmodel'<name>'hasmorethan160bones
```

This has a brutal implication:

You cannot solve this by simply “including all the BO3 bones.” A literal BO3 combined first-person rig is not a valid T6 end state.

8 Why the current approach fails

8.1 Failure A: wrong registration timing

The current script tries to perform zombies-weapon registration from inside the player-spawn loop. That is too late relative to how BO2 zombies initializes custom weapons.

What your code is really doing is:

- adding script-side metadata after the zombies weapon system has already gone through its normal init pass
- hoping that later table mutation will retroactively create a fully valid engine-side weapon registration

That is not a safe assumption, and it matches the history of `weapondef\_unregistered` behavior already seen in the repo.

8.2 Failure B: mismatched rig contract

The current viewmodel uses `viewmodel\_hands\_no\_model`. In T6, that implies the gun model itself must satisfy the first-person skeleton, hands, and tag contract needed for animation playback and camera stability.

But your current `bo3\_rev\_idg\_view` does not have:

- the full arm and finger chain expected by the BO3 anims
- the camera tag chain expected by T6
- the left and right weapon tags expected by the BO3 source-side authoring

So the model is being asked to do a job it simply cannot do.

8.3 Failure C: literal BO3 animation import is not a valid endgame

Even if you fixed the tag omissions, the full BO3 animation contract still exceeds T6’s first-person budget. This means the true solution is not “import more of BO3.”

The true solution is:

re-author or retarget the asset onto a BO2-compatible reduced rig that preserves the important behavior while staying under the T6 limit

8.4 Failure D: the current weapon is still semantically ray-gun-derived

The current fresh-name probe still inherits a ray-gun-family surface:

- ray gun ammo pool
- ray gun world model
- ray gun projectile and explosion FX
- ray gun ancestry in `parentWeaponName`

This is not wrong for a compatibility probe. In fact it is smart. But it means the probe should be treated as a **registration experiment**, not as proof that the real Apothicon Servant asset stack is almost working.

8.5 Failure E: not enough runtime proof for the current fresh-name path

One subtle but important point from the local logs: the older runtime evidence strongly supports the historical carrier and registration issues, but there is not yet strong fresh log evidence proving the current `apothicon\_servant\_zm` spawn-time path made it far enough to isolate the new failure cleanly.

That means the current repo state already contains enough evidence to reject the architecture, but it does **not** yet give you a final, clean, single-run proof of the fresh-name path’s exact failure point.

9 The correct mental model

9.1 You are solving three separate gates

You should think of the weapon port as three gates, in order:

1. **Registration gate:** can T6 zombies accept a fresh-name weapon early enough that `giveweapon()` sees it as valid?
2. **Rig gate:** can the first-person model satisfy T6's tag, camera, and bone-budget rules?
3. **Parity gate:** can the weapon then be made to behave and look like the BO3 IDG?

The mistake in the current work is trying to push all three gates at once.

9.2 What donor surfaces are actually for

A donor surface is useful because it lets you isolate variables.

If you borrow a ray gun surface, then a clean experiment is:

- keep the donor animations
- keep the donor rig expectations
- change only the registration strategy

If that fails, the problem is registration.

Then a second clean experiment is:

- keep donor gameplay and donor animations
- swap only the viewmodel mesh

If that fails, the problem is the model contract.

Your current probe mixes in BO3 animations too early, so it muddies the result.

9.3 What `viewmodel_hands_no_model` really implies

This field is not just a performance detail. In practice it says:

Do not attach a separate hand model; the viewmodel I am giving you already carries the hand-side contract.

That is why it is dangerous to use `viewmodel\_hands\_no\_model` with a weapon-only or mostly-weapon-only imported model. The engine will accept the field, but the animation and camera assumptions become your responsibility.

10 What you are doing wrong today, stated bluntly

1. You are trying to solve an **init-time weapon-registration problem** from **spawn-time GSC**.
2. You are treating `level.zombie\weapons["apothicon\_servant\_zm"]` as if it were equivalent to full engine registration.
3. You are feeding BO3 IDG animations to a model that is missing 110 bones from the animation union.
4. You are using `viewmodel\_hands\_no\_model` without providing a valid T6 combined first-person rig.
5. You are implicitly pursuing BO3 parity before proving BO2 acceptance.
6. You are still thinking of this as one bug, when it is at least two major incompatibility classes.

11 What the feasible path forward looks like

11.1 Short-term experiment plan

Experiment	What changes	Why	Pass condition
E1: Fresh-name registration only	Register <code>apothicon_servant_zm</code> during early map init, but keep pure donor ray-gun model and donor ray-gun anims.	Isolates lifecycle and registration from art and animation.	<code>giveweapon()</code> and <code>switch</code> succeed with a pure donor surface.
E2: Mesh acceptance only	Keep donor rig and donor anims; swap in an IDG-derived mesh that was explicitly rebuilt to satisfy the donor rig contract.	Tests whether the imported mesh is acceptable without mixing in BO3 animation semantics.	Weapon grants and switches, camera remains stable.
E3: Reduced-rig animation path	Build a BO2-compatible reduced combined rig with required tags and under-budget bones; retarget essential IDG states onto it.	This is the real bridge between T7 source art and T6 runtime.	First-person animation plays without missing-bone or camera-basis failure.
E4: Real behavior and FX	Replace donor projectile, trail, muzzle, and explosion behavior with IDG-specific equivalents as far as BO2 supports them.	Moves from acceptance to parity.	Visual and game-play behavior approximate the real weapon.

11.2 What the final architecture should probably be

The most realistic end-state architecture is:

1. **Early registration hook.** Use a custom script `main()` and, if needed, `replaceFunc()` or a full script-path override so that the zombies weapon addition happens before stock weapon initialization finishes.⁹
2. **Reduced BO2-compatible combined rig.** Preserve only the essential first-person tags and the bones needed for the visible behavior.
3. **Retargeting, not literal import.** Retarget the IDG to that reduced rig. Do not try to play the native BO3 `vm\_zod\_id\_gun\_*` set on the current imported model.
4. **Parity phase after acceptance.** Once grant, switch, and stable first-person presentation work, then move on to projectile/FX/script parity.

11.3 Why your repo is already pointing in this direction

The existence of `\_build/rebuild\_viewmodel\_glb.py` is important. That script is conceptually much closer to the correct end-state because it is about synthesizing or repairing a BO2-style first-person skeleton with the right tags and camera chain.

That is a better direction than the current probe path that simply copies `idg\_view.glb` and hopes the native BO3 animation set will line up.

12 Recommended implementation philosophy

If you want this project to become understandable instead of chaotic, keep these rules:

1. Never test registration and animation changes in the same run unless registration is already proven.
2. Never interpret “weapon asset exists in FF” as proof that `giveweapon()` should work.
3. Never interpret “animation compiled” as proof that the model is valid for it.
4. Treat camera tags as hard structural requirements.
5. Treat bone budget as a design constraint, not as an error to be bypassed later.

13 Summary

You are not failing because you are missing some obscure single trick. You are failing because the current probe crosses incompatible contracts.

The fresh-name BO2 zombies weapon problem needs to be solved at early init.

The BO3 IDG first-person asset problem needs to be solved by retargeting or rebuilding onto a BO2-compatible reduced rig.

Those are separate tasks. Once you separate them, the work becomes understandable:

⁹<https://plutonium.pw/docs/modding/gsc/new-scripting-features/>

- registration first
- rig acceptance second
- BO3 parity third

That is the architecture you should carry forward.

A Local repository evidence used for this analysis

File	Why it matters
mods/bo3_rev/scripts/mod_i_am_mod.gsc	Shows that the current weapon registration attempt happens on <code>spawned_player</code> , not during early zombies weapon initialization.
_build/build_bo3_rev_idg_probe.py	Shows that the fresh-name probe is a ray-gun-derived surface with IDG model and IDG animation references.
_build/bo3_rev_idg_probe/build_report.json	Confirms what assets are actually being packaged into <code>mod.ff</code> and <code>mod_load.ff</code> .
_build/bo3_rev_idg_probe/weapons/apothicon_servant_zm	Shows the actual staged weapondef-like raw data, including <code>handModel</code> , <code>parentWeaponName</code> , ray-gun ammo linkage, and BO3 animation references.
zone_dump/zone_raw/zm_tomb/weapons/ray_gun_zm	Gives the stock BO2 donor reference used to understand what fields are inherited versus newly changed.
decompiled_scripts/ZM/Core/maps/mp/zombies/_zm_weapons.gsc	Shows where zombies weapon initialization really happens in stock T6.
decompiled_scripts/ZM/Maps/Tranzit/maps/mp/zm_transit.gsc	Shows how a map script wires custom weapon callbacks before stock weapon setup finishes.
docs/explanation/constraints-and-barriers.md	Captures the T6 first-person bone budget and camera/tag contract findings.
docs/explanation/history-and-findings.md	Captures the previous registration, lane, bone, and camera-failure history.
_tmp_T6_Assets.h and _tmp_ContentLoaderT6.cpp	Show that T6 weapon loading and weapon-definition structure are deeper than a single string name or a single script table entry.
_build/rebuild_viewmodel_glb.py	Indicates that the correct direction is BO2-compatible rig reconstruction or synthesis, not raw BO3 model reuse.

B External references

1. Plutonium docs, *Loading Mods into Plutonium*: <https://plutonium.pw/docs/modding/loading-mods/>
2. Plutonium docs, *New Scripting Features*: <https://plutonium.pw/docs/modding/gsc/new-scripting-feat>
3. Plutonium changelog: <https://plutonium.pw/docs/changelog/>

4. OpenAssetTools, *What is OAT?*: <https://openassettools.dev/guide/what-is-oat.html>
5. OpenAssetTools, *Zone Files*: <https://openassettools.dev/reference/zone-file.html>
6. CoDMayaTools project page: <https://theenbywitch.github.io/CoDMayaTools/>
7. HydraX project page: <https://github.com/Scobalula/HydraX>